

La división más equilibrada sería esta:

Persona 1: AWS + CI/CD

Responsable de toda la infraestructura backend y despliegue productivo.

Tareas

- Optimizar y validar los Dockerfiles de los cuatro microservicios.
- Crear infraestructura en `infrastructure/`:
 - ECR.
 - Lambda.
 - API Gateway.
 - IAM.
 - CloudWatch.
 - CodeDeploy.
- Publicar las imágenes Docker en Amazon ECR.
- Configurar las cuatro funciones Lambda.
- Implementar API Gateway con:
 - Rutas `/v1/emergencies`, `/v1/dispatches`, `/v1/zones`, etc.
 - Stage `prod`.
 - CORS para el dominio de Vercel.
 - Validación de esquemas.
 - Rate limiting y throttling.
- Implementar recuperación dinámica de secretos desde:
 - AWS Secrets Manager o Parameter Store.
 - Roles IAM de mínimo privilegio.
- Eliminar credenciales por defecto de `config.py`.
- Crear pipeline CI/CD en `.github/workflows/` o GitLab:
 - Ejecutar pruebas.
 - Construir imágenes.
 - Publicar en ECR.
 - Desplegar Lambda.
 - Ejecutar rollback si falla.
- Implementar despliegue Canary con:
 - Lambda Aliases.
 - Alias `prod`.
 - CodeDeploy.
 - Tráfico progresivo.
 - Alarmas CloudWatch.
 - Rollback automático.
- Configurar métricas de:
 - Errores Lambda.
 - HTTP 5xx.
 - Latencia superior a 1500 ms.
- Configurar AWS Budgets:

- Presupuesto mensual de máximo 10 USD.
 - Alerta al superar 50 %.
 - Alerta proyectada al superar 85 %.
- Obtener y documentar:
 - URL de API Gateway.
 - Evidencia de Canary.
 - Evidencia de rollback.
 - Capturas de CloudWatch y AWS Budgets.

Archivos principales

```
infrastructure/
.github/workflows/
services/*/Dockerfile
services/*/app/config.py
gateway/
```

Recomendaría elegir la opción de **Canary con CodeDeploy**, porque el repositorio ya tiene preparados los handlers de Lambda y los targets `lambda` en los Dockerfiles.

Persona 2: Supabase + Vercel

Responsable de la base de datos productiva, autenticación y frontend.

Tareas de Supabase

- Recuperar y subir la carpeta `database/`.
- Eliminar `database/` de `.gitignore`.
- Agregar:

```
database/migrations/
database/seeds/
database/rls/
database/apply.sh
```

- Crear o configurar el proyecto Supabase.
- Habilitar PostGIS.
- Aplicar las migraciones y seeds.
- Configurar los esquemas:
 - `intake`
 - `dispatch`
 - `geo`
 - `notification`
- Crear autenticación con Supabase Auth.
- Implementar roles:

- CITIZEN
 - OPERATOR
- Crear políticas RLS para:
 - Emergencias propias del ciudadano.
 - Acceso completo del operador.
 - Recursos y despachos.
 - Notificaciones.
- Activar Supabase Realtime para:
 - intake.emergencies
 - notification.notifications
- Sustituir el polling de cinco segundos por suscripciones Realtime.
- Crear docs/seguridad.md.
- Verificar que la llave service_role nunca llegue al frontend.

Tareas de Vercel y frontend

- Instalar e integrar @supabase/supabase-js.
- Reemplazar el /login placeholder por autenticación real.
- Proteger la ruta /operator.
- Diferenciar las vistas de ciudadano y operador.
- Implementar PWA:
 - manifest.json.
 - Service worker.
 - Funcionamiento offline.
 - Almacenamiento local de reportes.
 - Sincronización al recuperar conexión.
- Agregar carga de evidencia fotográfica para daños estructurales.
- Configurar el despliegue en Vercel desde la rama principal.
- Configurar únicamente variables públicas en Vercel:
 - URL pública de Supabase.
 - Anon key.
- Obtener la URL de producción del frontend.
- Probar las cuatro ciudades:
 - Chocó.
 - Pereira.
 - Cali.
 - Manizales.

Archivos principales

database/
 frontend/
 supabase/
 vercel.json
 docs/seguridad.md
 .gitignore

Trabajo compartido

Ambas personas deben coordinar estos puntos:

- Definir nombres exactos de variables y endpoints.
- Mantener los enums y contratos de API sin cambios incompatibles.
- Probar la integración completa:

Frontend Vercel



API Gateway



Lambda



Supabase

- Crear los diagramas C4:
 - Contexto.
 - Contenedores.
 - Componentes.
 - Flujo de despliegue Canary.
- Ejecutar pruebas end-to-end en producción.
- Verificar los cuatro tipos de emergencias.
- Preparar el informe técnico.
- Grabar el video de máximo cinco minutos.
- Reunir las evidencias y capturas.
- Revisar que no existan secretos en ningún commit.

Orden recomendado

Primero

La Persona 2 debe restaurar `database/`, porque actualmente el repositorio clonado no puede ejecutar correctamente `docker compose up`.

Después, en paralelo

- Persona 1: AWS, ECR, Lambda y CI/CD.
- Persona 2: Supabase, RLS, Auth, Realtime y frontend.

Al final

- Persona 1 conecta API Gateway con la URL de Vercel.
- Persona 2 configura esa URL en CORS y Vercel.
- Ambos prueban el flujo completo y preparan las evidencias.